

Scaling, Parameter-Efficient Fine-Tuning, and KV-Cache Inference in Tiny Decoder-Only Language Models: An Empirical Study

Adyaan Ahmed

School of Electrical Engineering and Computer Sciences (SEecs)

National University of Sciences and Technology (NUST)

Islamabad, Pakistan

aahmed.bsds23seecs@seecs.edu.pk

Abstract—Scaling laws, low-rank adaptation (LoRA), and key-value (KV) caching are well studied in large language models, but whether their benefits carry over to models with only one to two million parameters is not obvious. I train three decoder-only Transformers (992K, 1.30M, and 1.73M parameters) from scratch on TinyStories with a custom byte-pair-encoding tokenizer, then use the largest model to compare LoRA against full fine-tuning, and measure generation latency with and without KV caching. Growing the model from 992K to 1.73M parameters lowers validation perplexity from 25.81 to 21.82 and raises top-1 accuracy from 29.04% to 32.07%, with training time growing more slowly than parameter count. LoRA cuts trainable parameters by roughly an order of magnitude but does not train faster than full fine-tuning in this setup, and full fine-tuning still reaches the lowest validation loss. KV caching gives inconsistent, mostly small latency changes (0.79x–1.19x) at these scales and generation lengths, averaging close to no benefit at all. The takeaway is that parameter efficiency, training-time efficiency, and inference-time efficiency are separate things that do not automatically move together once model size drops this low.

Index Terms—Tiny language models, decoder-only Transformers, LoRA, parameter-efficient fine-tuning, KV cache, inference efficiency, empirical study.

I. INTRODUCTION

Small language models trained on narrow, simplified corpora are a cheap setting for studying language model behavior. TinyStories [1] showed that models far smaller than production systems can still produce coherent text when trained on a restricted vocabulary and simple narrative structure. I refer to this regime loosely as TinyLMs: decoder-only Transformers on the order of one to a few million parameters, small enough to run controlled experiments repeatedly on a laptop.

A separate line of work studies efficiency at the other end of the scale: scaling laws that predict how loss falls with model and data size [2], [5], parameter-efficient fine-tuning methods such as LoRA [3], [4], and KV caching, which avoids recomputing attention keys and values at every generation step [6], [9]. These techniques are motivated by the cost of training and serving models with hundreds of millions to hundreds of billions of parameters. Whether they behave the same way at the opposite end of the scale is a separate question: per-layer computation is small, fixed overheads (Python dispatch,

memory allocation, framework bookkeeping) make up a larger share of total cost, and the trainable-parameter counts involved are already tiny in absolute terms.

This paper does not propose a new method. It asks a narrower empirical question: applied to TinyLMs under a controlled setup, do scaling, LoRA, and KV caching behave the way their large-scale reputation would predict?

RQ1. Does increasing model capacity from about 1M to 1.7M parameters consistently improve language-modeling performance?

RQ2. How does LoRA compare with full fine-tuning when adapting a small pretrained model, in trainable parameters, validation performance, and training time?

RQ3. Does KV caching give meaningful end-to-end generation speedups for TinyLMs, and how does that vary with model size and generation length?

I answer these with three controlled experiments on models I train and evaluate myself, sharing a tokenizer, data split, and evaluation protocol throughout. The contribution is empirical, not algorithmic: parameter efficiency, training-time efficiency, and inference-time efficiency turn out to be distinct axes that do not necessarily move together at this scale. In particular, LoRA’s large cut in trainable parameters does not translate into a training-time speedup here, and KV caching’s benefit at these scales and generation lengths is small and inconsistent rather than absent or reliably large.

II. BACKGROUND

My model follows the standard decoder-only, GPT-style design [7], [10]: token embeddings plus learned positional embeddings, L Transformer blocks with causal multi-head self-attention and a position-wise feed-forward network, pre-normalization, and residual connections:

$$h' = h + \text{Attn}(\text{LN}(h)) \quad (1)$$

$$h'' = h' + \text{FFN}(\text{LN}(h')) \quad (2)$$

with causal, scaled dot-product attention

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \quad (3)$$

where M masks out future positions. The output head is tied to the input embedding table, halving the embedding-related parameter count. None of this architecture is a contribution; it just establishes what I measure.

LoRA [4] freezes a pretrained weight matrix $W_0 \in \mathbb{R}^{d \times k}$ and represents the update as a low-rank product $W = W_0 + \frac{\alpha}{r}BA$, with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and $r \ll \min(d, k)$. Only A and B train; W_0 stays fixed. Crucially, LoRA does not by itself reduce the computation performed through W_0 during the forward or backward pass – the frozen matrix still takes part in the full matrix multiplication, and the low-rank path adds computation on top of it. That distinction is the crux of Experiment 2.

KV caching stores the key and value tensors from previous positions during autoregressive generation so only the newest token’s key and value need computing at each step [6], [9]. This removes redundant computation but adds memory and bookkeeping cost, and whether the trade is worth it depends on how large the avoided computation is relative to that overhead – the question Experiment 3 asks directly.

III. RELATED WORK

The Transformer [10] and its decoder-only, GPT-style variant [7] underlie most modern language models, including the ones here. Kaplan et al. [5] showed loss follows power laws in model size, data size, and compute; Hoffmann et al. [2] argued model size and data should scale together. Both operate many orders of magnitude above my setting – I do not test a scaling law, only whether performance improves monotonically over the narrow range I could afford to train. Adapter modules [3] and LoRA [4] were proposed to adapt large pretrained models without updating every parameter, motivated by the storage and deployment cost of many full fine-tuned copies; reported results at large scale show competitive quality alongside large parameter savings and, in some settings, better training throughput [4]. Experiment 2 checks whether the parameter-efficiency side of that trade-off shows up at TinyLM scale, and separately whether it comes with a training-time benefit. KV caching and related ideas for serving large models efficiently are well covered elsewhere [6], [9], but that literature assumes large models, long contexts, and high serving throughput – all different from the short generations studied here. TinyStories [1] showed models under ten million parameters can produce coherent short stories on a simplified corpus; I build on that setting but ask a different question, about training and inference efficiency rather than generation quality. None of this work tests scaling, LoRA, or KV caching directly at the million-parameter scale with short generations, which is the gap this paper fills empirically.

IV. EXPERIMENTAL SETUP

Dataset. I use TinyStories [1], taking a fixed, unshuffled prefix of the official splits: 8,000 training stories and 2,000 validation stories, against a full corpus of roughly 2.12M training and 21,990 validation stories (about 0.38% and 9.1% of the

full splits). Training and validation data never mix. After tokenization the training split is 1,952,319 tokens and validation is 450,301 tokens. Each story is Unicode-normalized (NFKC), stripped of non-printable characters, whitespace-collapsed, and trimmed; nothing is truncated, filtered, or deduplicated.

Tokenizer. I trained a byte-pair-encoding tokenizer [8] from scratch in pure Python, learning 910 merges on top of 89 single-character tokens for a 1,000-token vocabulary (999 learned tokens plus `<unk>`). Nothing about whitespace or punctuation is special-cased – every character starts as its own token and is merged purely by frequency. This vocabulary is small next to production tokenizers (tens of thousands of tokens); I return to what that might mean in Section IX. The same tokenizer is used everywhere in the paper, so it is never a confound between conditions. As a sanity check, a unigram frequency baseline reaches a validation perplexity of 589.63 under this vocabulary; every trained model in this paper does far better, which at least confirms they are learning something beyond token frequency.

Model and training. All models share the architecture in Section II, with size varied by layer count, head count, and embedding dimension (see Table I for the exact configurations). Training uses a context length of 128 tokens, batch size 32, AdamW (default betas and epsilon, weight decay 0.01, constant learning rate 3×10^{-4} , no schedule), and dropout 0.1 with no gradient clipping. Experiment 1 trains each model for 3,000 steps, sampling a fresh random 128-token window per step. No random seed is set anywhere in training or data sampling (a seed is used only in an unrelated smoke test described below), so every run here, including repeated configurations, reflects whatever the ambient random state happened to be. I treat this as a limitation (Section IX) rather than something to paper over.

I report validation cross-entropy loss, perplexity (e^{loss}), and top-1/top-5 next-token accuracy, computed over the entire validation split rather than a sampled batch, so the numbers are deterministic for a given checkpoint.

Hardware and timing. Everything ran on one machine: an Apple M1 Pro MacBook Pro, 16GB RAM, PyTorch 2.4.0 on the MPS backend. For the KV-cache comparison, each timed result excludes one warmup generation and averages three repeats, with `torch.mps.synchronize()` called before each clock read so queued MPS work cannot leak across the timer boundary. The latency benchmark uses a 10-token prompt and generates 20, 50, or 100 tokens with sampling (temperature 0.8, top- k 50) in both the cached and uncached paths. A separate smoke test confirms the two paths produce identical sampled tokens given the same seed, so the timing comparison isolates speed rather than measuring a difference in what was generated. Peak memory was not measured – I had no profiling instrumentation in place, which I flag rather than hide.

Fair-comparison controls. Every experiment shares a tokenizer and data split. Experiment 1 holds the training configuration fixed across model sizes so only capacity varies. Experiment 2 starts every variant, full fine-tuning included,

TABLE I
MODEL SCALING RESULTS. ALL MODELS TRAINED FROM SCRATCH
UNDER AN IDENTICAL TRAINING CONFIGURATION.

	Small	Medium	Large
Layers	4	6	8
Heads	4	6	8
Embed dim	132	126	128
Parameters	992,376	1,295,280	1,730,816
Val. loss	3.2508	3.1576	3.0827
Val. PPL	25.81	23.51	21.82
Top-1 acc.	29.04%	30.82%	32.07%
Top-5 acc.	55.52%	57.37%	58.79%
Train time	265.6s	334.0s	407.3s

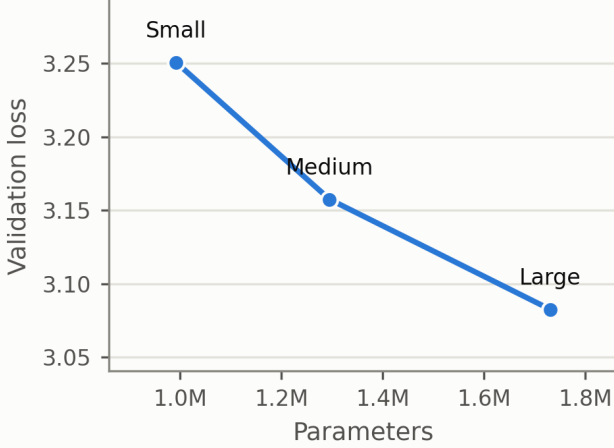


Fig. 1. Validation loss decreases monotonically as parameter count increases from 992K to 1.73M.

from an identical deepcopy of the same trained 1.73M-parameter checkpoint, so all variants begin from bit-identical weights and train for the same 500 steps. One control does *not* hold: because no seed is set, the exact batch sequence differs across Experiment 2 variants, adding some irreducible run-to-run noise on top of the deliberate variable (how many parameters are trainable).

V. EXPERIMENT 1: MODEL SCALING

I train three Transformers from scratch, varying depth, heads, and embedding dimension to span roughly 992K to 1.73M parameters (Table I).

Both loss and perplexity fall monotonically across all three sizes (Fig. 1, Fig. 2), and top-1/top-5 accuracy rise monotonically alongside them – no reversal or plateau anywhere in the range I tested. Going from Small to Medium (+30.5% parameters) cuts perplexity 8.9%; Medium to Large (+33.6% parameters) cuts it a further 7.1%, a similar relative step each time, hinting at (though far too short a range to confirm) diminishing returns rather than a sharp threshold.

Training time rises too, from 265.6s to 407.3s (+53.3%), but more slowly than parameter count (+74.4% over the same range). I would not read too much into that sublinearity: the

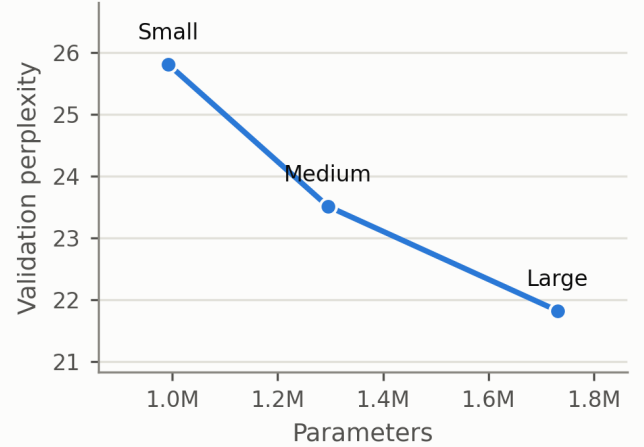


Fig. 2. Validation perplexity decreases monotonically as parameter count increases.

three configurations differ in depth and head count together, and embedding dimension is not even monotonic across them (132, 126, 128, solved per target parameter count rather than held fixed), so more than raw parameter count is driving training time.

For RQ1, then: within the tested range, more parameters helped on every metric, every time, with no exceptions. Whether the extra cost is worth it depends on the application – the Large model costs 53% more time to train than Small for a 15.5% relative perplexity reduction and a 3-point top-1 accuracy gain – and that trade-off is really a question for whoever is deploying the model, not something this paper can settle in general. Three points spanning less than a factor of two, with no seed control, is a narrow basis for any broader scaling claim (Section IX).

VI. EXPERIMENT 2: LoRA VS. FULL FINE-TUNING

I take the 1.73M-parameter model from Experiment 1 as a base and compare the untouched baseline, LoRA at ranks 4, 8, and 16, and full fine-tuning, each for 500 steps. LoRA wraps all four linear layers per block (the fused QKV projection, the attention output projection, and both feed-forward layers), with $\alpha = 2r$ so α/r stays fixed at 2.0 across ranks and only capacity varies. LoRA dropout is 0. Base weights are frozen by construction (`requires_grad=False`), and every variant, including full fine-tuning, starts from an identical deepcopy of the same checkpoint and uses the same optimizer settings as Experiment 1.

LoRA does what it says on the parameter side: rank 4 trains only 3.65% of what full fine-tuning trains, about a 26x reduction (Fig. 3), and it improves over the baseline even at that smallest rank (perplexity 21.82 $\rightarrow$ 20.80). Going from rank 4 to 16 helps a little more (20.80 $\rightarrow$ 20.67 $\rightarrow$ 20.48, Fig. 4), but the gain per doubling of rank is small. Full fine-tuning still wins on validation performance outright, at 19.57 – 4.4% below the best LoRA rank I tried.

TABLE II
LoRA VS. FULL FINE-TUNING, ADAPTING THE 1.73M-PARAMETER BASE MODEL.

Variant	Train. par.	%	Val. PPL	Time
Baseline	0	0.00%	21.82	0.0s
LoRA $r=4$	65,536	3.65%	20.80	123.9s
LoRA $r=8$	131,072	7.04%	20.67	116.8s
LoRA $r=16$	262,144	13.15%	20.48	111.1s
Full FT	1,730,816	100%	19.57	81.8s

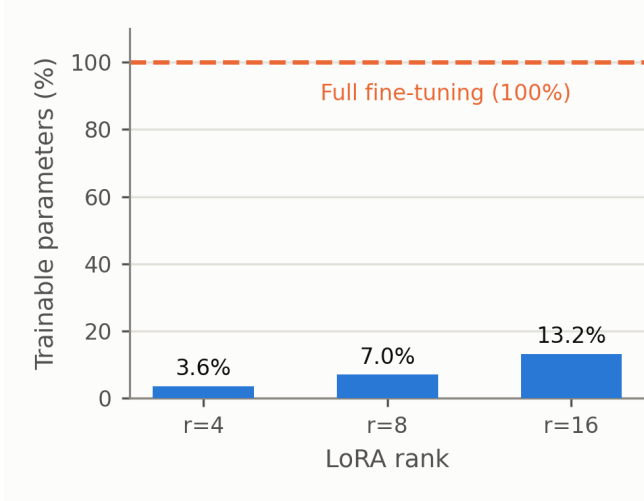


Fig. 3. Percentage of total parameters that are trainable, by LoRA rank, relative to full fine-tuning (100%).

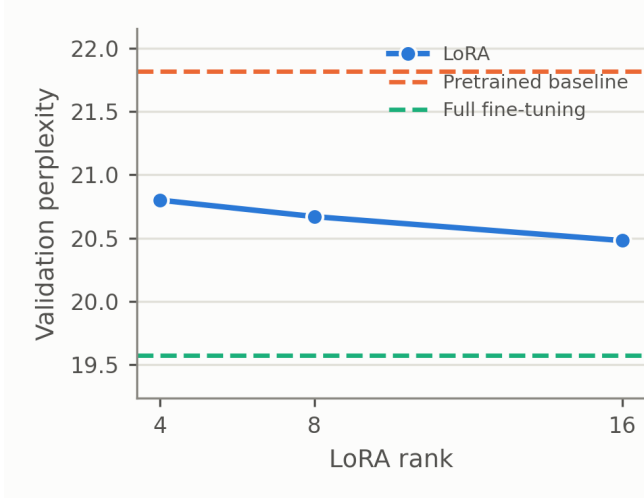


Fig. 4. Validation perplexity as a function of LoRA rank, with the pretrained baseline and full fine-tuning shown for reference.

The odd result is training time. Every LoRA rank was *slower* in wall-clock terms than full fine-tuning – 123.9s, 116.8s, and 111.1s for ranks 4, 8, and 16, against 81.8s for full fine-tuning, despite training 7–26x fewer parameters. This is not a general property of LoRA so much as a consequence of what freezing actually does: a frozen weight still takes part in the full forward and backward matrix multiplications for its

TABLE III
KV-CACHE SPEEDUP (NO-CACHE TIME / CACHED TIME) BY MODEL SIZE AND GENERATION LENGTH.

Model	20 tok.	50 tok.	100 tok.
Small (992K)	0.85x	0.96x	0.97x
Medium (1.30M)	1.07x	0.79x	0.95x
Large (1.73M)	0.97x	1.06x	1.19x

layer, so the compute those layers require does not shrink just because gradients are not computed for them. LoRA’s low-rank path then adds computation on top of that unchanged forward pass. At this model’s scale, whatever backward-pass work is saved by freezing most of the network is apparently not enough to offset the low-rank path and other fixed overhead in my implementation. I also notice training time falling slightly as rank rises (123.9s $\rightarrow$ 111.1s), which runs against the expectation that a larger rank should cost more – each of these numbers is a single run with no seed control, so I treat that particular ordering as suggestive rather than something to build a claim on.

For RQ2: LoRA delivers on parameter efficiency and recovers most of the quality gap to full fine-tuning (rank 16 closes about 58% of the perplexity gap using 13% of the parameters), but it does not train faster here – if anything, the opposite. Fewer trainable parameters and faster training are easy to conflate, and at this scale they are not the same thing. I am not claiming LoRA is slower in general, only that in this setup, on this hardware, the training-time benefit people often assume did not show up, and should be measured rather than assumed at small scale.

VII. EXPERIMENT 3: KV-CACHE INFERENCE

I measure end-to-end generation latency for all three Experiment 1 models, generating 20, 50, and 100 tokens from a 10-token prompt, with and without KV caching, using the same sampling settings as before. Table III and Fig. 5 report the speedup (no-cache time divided by cached time); above 1.0 means caching won.

Of nine configurations, caching won in four and lost in five; the best case was 1.19x (Large, 100 tokens), the worst 0.79x (Medium, 50 tokens), and the mean across all nine sits at roughly 0.98x – essentially no net benefit. The Small model never benefited, at any length tested. Medium was non-monotonic (a small win at 20 tokens, a clear loss at 50, near parity at 100). Large was the only model with a clean trend, improving as generation length grew, and the only one where caching’s edge exceeded about 7%.

I do not read this as KV caching being useless for TinyLMs – only that at these sizes and these generation lengths, on this hardware, it did not reliably pay off. A few explanations seem plausible, though I have not isolated which dominates: the attention computation caching avoids is small in absolute terms at this scale, so cache bookkeeping (allocating, indexing, appending at every step) may be comparable in size to what it saves rather than negligible next to it; generation lengths up to

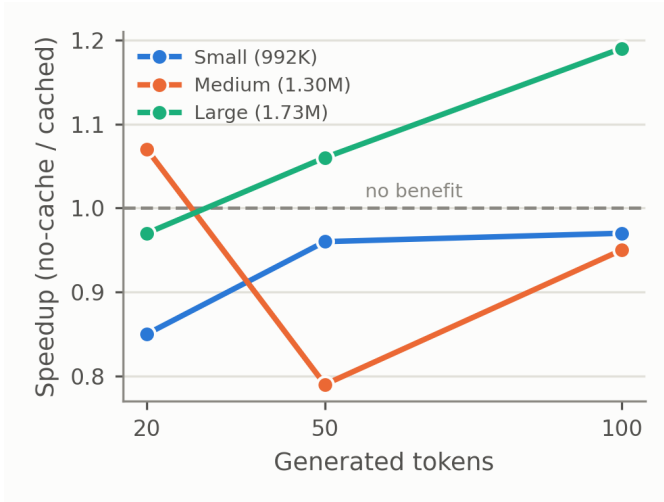


Fig. 5. KV-cache speedup vs. generation length, by model size. Values below the dashed line indicate cached generation was slower than uncached generation.

100 tokens may simply be too short to reach the regime where caching’s saved computation outweighs its overhead; and fixed per-step framework and MPS-backend overhead may dominate the time budget regardless of caching. That the largest model and longest generation length show the clearest benefit is at least consistent with all three explanations, but I did not run the ablation needed to separate them, nor test a non-MPS backend to see if this is backend-specific.

VIII. DISCUSSION

The three experiments point at one idea: efficiency is not one thing. I saw at least four separate axes across these experiments – parameter count, training time, inference time, and model quality – plus a fifth, memory, that I simply could not measure without profiling tools I did not have. Scaling improved quality on every metric but cost more training time to get there. LoRA improved parameter efficiency sharply and kept most of full fine-tuning’s quality gain, but made training slower rather than faster. KV caching is built specifically to help inference time, and at this scale it mostly did not, losing in five of the nine configurations tested.

The LoRA training-time result feels like the one worth remembering, because it is the one people are likeliest to assume rather than check. Fewer trainable parameters does not mean less computation: freezing a weight removes its gradient update, not its participation in the forward and backward matrix multiplications. Whether that saving beats LoRA’s added low-rank computation is a question of scale, not a fixed property of the method. The KV-cache results make a related point from the inference side – an optimization that removes redundant computation only pays off when that computation is large relative to what is left, and at TinyLM scale with short generations, it often is not.

None of this is a claim that LoRA is slower in general, or that KV caching does not help small models – only that, in

this specific controlled setup, the benefits usually assumed by default did not appear. Techniques validated at large scale are worth re-measuring, not just re-using, when the target scale is this different.

IX. LIMITATIONS

The biggest one is scale: Experiment 1 covers only three model sizes across less than a 2x range in parameter count, differing simultaneously in depth, head count, and embedding dimension (which is not even monotonic across the three: 132, 126, 128). That rules out any precise scaling-law claim; it only tells me that, in this narrow range, more parameters helped and never hurt. The 1,000-token vocabulary and the 8,000/2,000-story training subset (well under 1% and about 9% of the full TinyStories splits, respectively) are both far smaller than typical setups, and results here may not hold at full scale. Every timing number comes from one machine (an M1 Pro on the MPS backend) and, more importantly, from a single, unseeded run per configuration – there is no averaging across independently seeded runs anywhere except the three within-run repeats used for the KV-cache numbers, so small timing gaps (especially between LoRA ranks) should be read as one-off measurements, not statistically robust differences. I also did not measure peak memory, test generation lengths past 100 tokens, compare against a non-MPS backend, or test model sizes above 1.73M parameters, so the LoRA and KV-cache findings are scoped strictly to what I tested here, not to TinyLMs or efficiency techniques generally.

X. CONCLUSION

Within the range tested, growing the model from 992K to 1.73M parameters helped on every metric, at a real but sublinear training-time cost. LoRA cut trainable parameters by up to 26x and recovered most of full fine-tuning’s quality gain, but trained slower rather than faster in wall-clock terms – the opposite of what its parameter savings might suggest. KV caching gave a small, inconsistent latency benefit that only became clear for the largest model at the longest generation length tested, and lost outright in over half the configurations tried.

None of this establishes universal rules about scaling, LoRA, or caching. It is a case for measuring efficiency claims in the regime where they will actually be used, rather than assuming they transfer down from the scale they were validated at. Extending this work would mean testing a wider parameter range, longer generation lengths, multiple seeded runs per configuration, other hardware backends, and direct memory measurement – all of which would help separate what is specific to this exact setup from what holds more generally at TinyLM scale.

REFERENCES

- [1] R. Eldan and Y. Li, “TinyStories: How small can language models be and still speak coherent English?” *arXiv preprint arXiv:2305.07759*, 2023.

- [2] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556*, 2022.
- [3] N. Hounsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *Proc. 36th Int. Conf. Machine Learning (ICML)*, 2019.
- [4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Int. Conf. Learning Representations (ICLR)*, 2022.
- [5] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.
- [6] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling Transformer inference," *arXiv preprint arXiv:2211.05102*, 2022.
- [7] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI Technical Report, 2019.
- [8] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proc. 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.
- [9] N. Shazeer, "Fast Transformer decoding: One write-head is all you need," *arXiv preprint arXiv:1911.02150*, 2019.
- [10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.